The University of Hong Kong

School of Computing and Data Science

COMP7705

Project Report

Semantic-Motion: A Multi-view Engineering Pipeline for Perception, Fact-preserving Language Augmentation and Fail-closed Multimodal Refinement

Submitted in partial fulfillment of the requirements for the admission to the degree of Master of Science in Computer Science

By

Gai Yifan(), Yang Tongzhou(), Li Chenyang (3036657835)

Mentor: Prof. Taku Komura

Date of submission: 17/07/2026

Declaration

State originality, academic integrity, use of generative AI where required, and individual contributions for a group project.

Abstract

Keep the abstract below 200 words. State the data problem, the four-module pipeline, the evaluated outputs, the strongest quantitative result, and the main limitation. Do not describe planned work as completed work.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Chapter 2

Technical Background and Related Systems

2.1 Vision-Language-Action Models

2.2 Robot Learning Benchmarks

2.3 Vision-Language Models for Robotics

2.4 Video-to-Task Generation

2.5 Demonstration Data Quality

2.6 Engineering Gap

Chapter 3

System Requirements and Overall Design

3.1 Inputs, Outputs and Use Cases

3.2 Design Goals

3.3 Overall Architecture

3.4 Module Responsibilities and Boundaries

3.5 Data Models and Interface Contracts

3.5.1 Paired Multi-view Video Contract

3.5.2 Shared Timebase and Motion Trajectory Contract

3.6 Observability and Visualisation Design

3.7 End-to-End Workflow

Chapter 4

Design and Implementation of the Four Modules

4.1 The Rendering Module

In the end-to-end data flow of the Semantic-Motion system, the Rendering Module serves as a critical bridge connecting raw motion data with Vision-Language Models (VLMs). Traditional Motion Capture data (such as FBX or BVH formats) essentially consists of a series of 3D coordinate point clouds and skeletal rotation matrices that change over time. Because general-purpose VLMs lack the capability to directly parse pure 3D geometric data, a significant ”representation gap” exists. The design objective of this module is to construct a fully automated, high-fidelity rendering pipeline that transforms abstract ”digital signals” into VLM-perceptible ”visual signals” (multi-view videos), while simultaneously exporting standardized 3D physical trajectories to provide high-quality data sources for downstream semantic perception and quality refinement. The core implementation of this module is built upon the Blender 5.1 Python API.

4.1.1 Motion Data Import and Scene Initialisation

To achieve zero-human-intervention processing for large-scale datasets, the rendering module first establishes a robust batch import and scene initialization mechanism. Upon pipeline initiation, the system iterates through the FBX or BVH motion files in the input directory, importing the armature and animation data via Blender's native interfaces. Addressing the issue of variable action lengths, the script automatically reads the `animation_data.action.frame_range` attribute of the armature to dynamically set the start and end frames of the current scene, thereby achieving adaptive animation durations.

In industrial-grade batch processing, memory leaks and scene pollution are common issues. To address this, the module implements a whitelist-based Trace-free Cleanup Mechanism. Before importing each new action, the system automatically traverses the current scene, destroying the temporary meshes and skeletal data generated in the previous iteration, while strictly preserving foundational scene objects such as lights, cameras, and the ground plane. This state isolation strategy avoids the overhead of repeatedly creating the base environment, significantly enhancing throughput. Furthermore, considering that external motion data may be corrupted, the import phase encapsulates a Try-Except fault-tolerance mechanism. When encountering fatal errors such as an ”invalid header,” the system automatically logs the error and skips the file, ensuring that the entire engineering pipeline does not crash due to a single corrupted data point.

4.1.2 Scene Context Generation

To assist the VLM in focusing on the action itself and accurately recognizing macro-intents, the module incorporates a dynamic Scene Context Generation algorithm based on semantic parsing and skele-

tal dynamic tracking. Pure actions (e.g., sitting down in mid-air) are prone to trigger VLM hallucinations or misjudgments. Therefore, by parsing keywords in the motion filenames and combining them with the skeletal coordinates in physical space, the system dynamically injects minimalist geometric anchors:

Vault/Step: When a vaulting or stepping semantic is identified, the system iterates through every frame of the animation to extract the highest point (peak of flight) of the Root/Hips bone along the Z-axis in the world coordinate system. Subsequently, the corresponding (X, Y) coordinates of that frame are projected onto the ground, and a rectangular cuboid obstacle is generated at this geometric center. This ensures that regardless of where the character initiates the jump, the obstacle perfectly aligns with its motion parabola.

Sit: The system shifts to tracking the lowest point of the Hips bone along the Z-axis and generates a cubic chair directly beneath it, resolving the clipping issue where the character appears to "sit in mid-air" during linear displacement.

Drop/Jump Down: For waterfall-type trajectories where the character drops downwards, the system reads the posture of the initial jump frame, iterates through bones such as the Ankle and Toe, and precisely locates the absolute lowest point of the sole. Simultaneously, the algorithm calculates the directional vector from the jump to the landing and applies a dynamic offset to the generated 2x2 meter platform in the opposite direction of the jump. This positions the character exactly at the edge of the platform in the initial frame, perfectly recreating the physical logic of "jumping down from a height."

4.1.3 Multi-view Video Synthesis

To meet the Semantic-Motion system's requirements for multi-granular semantic extraction, the rendering module deploys a dual-track virtual camera system within the 3D engine to simulate a real-world, integrated hardware-software capture environment:

Fixed View: Deployed globally within the scene, this view is utilized to capture the character's full-body kinematics, movement trajectories, and macro-spatial relationships with the environmental context.

Ego View: Aimed at simulating the robot's head-mounted perception. The script uses Python to traverse and lock onto key bones containing "Head" in their names, dynamically binding the camera to the character's head via the `parent_bone` attribute. Subsequently, the viewing direction is corrected using Euler Rotation, enabling it to track the character's line of sight with zero distance. This perspective is crucial for the downstream Perception Module to capture micro-action details, such as Hand-Object Interactions (HOI).

On the rendering output level, the system invokes the Blender EEVEE rendering engine to automatically overwrite the environment background with a solid color (mid-gray/black), eliminating ambient light noise. The output format is configured as a 1024×1024 resolution H.264 encoded MP4 video, ensuring that the VLM is provided with high-definition, interference-free visual inputs even under lower VRAM constraints.

4.1.4 3D Motion Trajectory Export

While generating visual signals, the rendering module is also tasked with extracting physical Ground Truths. Pure video data cannot directly provide precise physical metrics (e.g., jump height, accelera-

tion), which are indispensable for validating motion quality.

This section embeds a trajectory export algorithm within the rendering loop. The algorithm forces an update of the view layer via `bpy.context.view_layer.update()` and reads the product of specific Pose Bones and the `matrix_world` frame-by-frame to obtain absolute physical coordinates. To ensure compatibility with heterogeneous motion capture standards (e.g., Mixamo, CMU), the system features built-in skeletal mapping dictionaries that automatically match and extract the spatial coordinates of key nodes like the Root (center of mass), Hand_R, and Hand_L. The data is truncated to four decimal places to reduce storage redundancy and serialized into JSON format. This design bifurcates the pipeline: the rendered videos flow into the Perception Module, while the precise 3D trajectory data flows directly into the downstream Refinement Module, providing numerical bases for the final Motion Quality Scoring.

4.1.5 Rendering Outputs and Manifest

Following the automated processing of the aforementioned engineering pipeline, the Rendering Module stably generates three core deliverables in the output directory for every inputted raw motion file (e.g., `drop_from_roof.fbx`):

- **Fixed_View.mp4**: A third-person global video featuring the scene context.
- **Ego_View.mp4**: A first-person detail video closely following the character's head movements.
- `<action_name>_trajectory.json`: A 3D motion trajectory file containing the frame-by-frame (X, Y, Z) coordinates of core bones.

These standardized outputs completely eliminate the representation barriers of the raw data, laying a solid underlying data foundation for the system's subsequent semantic generation and multimodal data refinement.

4.2 The Perception Module

4.2.1 Paired-view Input and Synchronous Frame Sampling

4.2.2 Overlapping Macro Windows and Boundary Voting

4.2.3 VLM-based Structured Scene Understanding

4.2.4 Macro-Intent Recognition

4.2.5 Dense Micro-Instruction Parsing

4.2.6 Body, Contact, Posture and Trajectory Evidence

4.2.7 Temporal Task Boundary Detection

4.2.8 Segment-level Temporal Aggregation

4.2.9 Static Perception Benchmark

4.3 The Fact-preserving LLM Augmentation Module

4.3.1 Input and Interface

4.3.2 Instruction Reconstruction

4.3.3 Augmentation Strategies

4.3.4 Deterministic Fact Preservation and Hallucination Control

4.3.5 Output and Provenance

4.4 The Refinement Module

4.4.1 Sample Construction

4.4.2 Motion Data Normalisation

4.4.3 Motion Quality Scoring

4.4.4 Semantic Consistency Verification

4.4.5 Deterministic Cross-modal Alignment Checks

4.4.6 Fail-closed Refinement Decision and Explainable Outputs

Chapter 5

System Integration, Demonstration and Evaluation

Chapter 6

Engineering Discussion

6.1 Interpretation of System Results

From the perspective of end-to-end execution, the system demonstrates exceptional capabilities in ”macro-task understanding,” yet reveals clear performance bottlenecks in ”micro-action and spatial parsing.” The Vision-Language Model (VLM) achieves a high accuracy rate in recognizing macro-intents (e.g., ”moving a box”), indicating that the large model's prior knowledge of general scenes generalizes well to the robotic rendering perspective. However, when processing spatial relationships and fine-grained operations (e.g., determining the exact distance and contact state between the end-effector and the object), the accuracy drops significantly.

Furthermore, the system outputs reveal a strong presence of cascading errors. A perceptual error or hallucination by the VLM in a single frame directly propagates into the temporal boundary detection of the task, leading to over-segmentation or erroneous merging of action segments. This explains why relying solely on vision-based large model predictions is insufficient, and it validates the absolute necessity of the dual-channel refinement mechanism (incorporating both ”motion trajectory quality” and ”semantic consistency”) in the Refinement Module. The dual-channel approach is significantly more effective at intercepting misleading data samples than relying on a single visual metric.

6.2 Engineering Strengths

The greatest engineering value of this project lies in the construction of a runnable, observable, and extensible closed-loop data infrastructure, rather than merely algorithmic fine-tuning:

Lightweight and Low Resource Dependency: The pipeline design drastically lowers the computational threshold. By calling cloud-based VLM APIs for visual understanding tasks, the system can run smoothly without the need for local high-performance GPU clusters.

High Modularity and Multi-source Compatibility: The four modules of the system are highly decoupled. This not only allows for seamless integration of more advanced VLMs or LLM rewriters in the future, but also enables the system to be compatible with highly divergent data sources. Whether processing raw FBX motion capture data with complex skeletal hierarchies or standardized LIBERO robotic trajectories, the system can clean and align them through unified data interfaces.

Strong Interpretability of Decisions: The automated data refinement process is not a black box. The output from the Refinement Module is not just a binary keep/drop decision; it includes detailed motion anomaly penalties (e.g., `high_jerk_spikes`) and semantic mismatch reason codes, providing solid data support for subsequent manual review and system debugging.

6.3 Failure Mode Analysis

During actual engineering execution, the system exposed several typical failure modes:

Visual Feature Confusion and Object Misidentification: The VLM often misidentifies interfer-

ing objects in the background as target objects, or experiences semantic confusion between visually similar components.

Information Loss due to Temporal Sampling: The uniform frame sampling strategy, adopted to balance cost and efficiency, may inadvertently miss critical interaction moments that last only for a fraction of a second, resulting in gaps in the generated micro-instruction descriptions.

Spatial Alignment and Rendering Biases: In the Rendering Module, when simple scene contexts (like obstacles or boxes in a vaulting action) are dynamically generated based on skeletal trajectories, spatial misalignments occasionally occur between the generated obstacle and the actual skeletal movement. This spatial dislocation in the rendered environment directly induces the VLM to produce false ”non-contact” or ”collision” hallucinations.

Boundary Collapse Triggered by Single-frame Hallucinations: Even if the action itself is continuous, if the VLM hallucinates in a single frame (e.g., falsely concluding that the target object has changed), the temporal segmentation algorithm will misjudge it as the start of a new task, leading to a logical fragmentation of the entire task sequence.

6.4 Limitations

Although Semantic-Motion provides a complete end-to-end pipeline, the current system is still constrained by several key factors:

Embodiment Gap: The Rendering Module currently processes human skeletal motion capture data. The degrees of freedom and kinematic characteristics of human motion possess an inherent embodiment gap compared to actual robotic arms (like UR5 or Franka). This implies that high-quality action descriptions refined based on human models may still require further alignment mapping before being directly transferred to robotic action generation models.

Lack of Large-scale Gold-label Data: The evaluation of Module C is currently limited by a small scale of video cases and static benchmarks (predominantly synthetic images). The lack of large-scale, human-expert annotated datasets for real robotic operations makes it difficult to rigorously prove the absolute performance gain of the dual-channel refinement on the final VLA training efficacy from a statistical standpoint.

Uncontrollability of External Dependencies: While heavily relying on cloud APIs brings convenience, it introduces risks regarding reproducibility. Silent updates to the underlying cloud models may result in different semantic outputs when feeding the exact same video input.

Limitations of Text Augmentation: Although the current rule-template-based Augmentation Module guarantees zero-hallucination, it sacrifices the rich diversity of natural language. It cannot generate instruction variants encompassing complex environmental modifiers or tonal shifts like a native LLM.

6.5 Engineering Trade-offs

The core process of building this system is essentially a collection of engineering trade-offs:

Convenience of Cloud APIs vs. Controllability of Local Deployment: Utilizing cloud APIs lowered the initial construction cost and hardware threshold but sacrificed local execution determinism and version-locked reproducibility.

Cost Advantage of Uniform Sampling vs. Temporal Information of Keyframes: Frame-by-

frame analysis carries an exorbitant computational overhead, making uniform sampling a necessary compromise. While it improves batch processing efficiency, it inevitably increases the risk of missing high-frequency motion details.

Safety of Template Augmentation vs. Diversity of LLMs: Considering the strict accuracy requirements for VLA instructions, we opted for a template-based rewriting strategy, sacrificing linguistic diversity in exchange for 100% semantic consistency.

Strict Aggregation (min) vs. Lenient Filtering (mean): When processing multi-trajectory motion scoring, using min aggregation maximizes the interception of anomalous trajectories (high precision), but it is also prone to incorrectly discarding high-value samples that contain minor, localized noise (low recall). This requires dynamic adjustment based on dataset scale during the actual construction of training sets.

6.6 Evaluation Validity

The validity of the current system evaluation methods can be examined across the following four dimensions:

Internal Validity: Through rigorous component ablation studies, we effectively validated the logical superiority of "dual-channel refinement" over "single-metric refinement."

External Validity: The current static benchmarks focus on synthetic images and five specific categories of robotic tasks. The system's generalization capabilities in handling extreme lighting conditions or open-vocabulary long-tail real-world scenarios require further empirical validation.

Construct Validity: Utilizing the ratio of jitter and jerk in motion trajectories as metrics for action data quality is theoretically sound in physics and control theory, effectively reflecting the smoothness and usability of the demonstration data.

Reproducibility: Although the code architecture is fully open-source and data formats are standardized, achieving completely consistent reproducibility across different time periods remains inherently challenging due to the reliance on external large model APIs.

6.7 Implications for VLA Data Pipelines

The engineering practice of Semantic-Motion offers several core insights for data engineering of future large-scale VLA models:

First, independent vision-language understanding validation is mandatory before feeding data into action generation models. Raw collected motion data contains significant amounts of unintentional wandering and trial-and-error; all trajectories cannot be blindly accepted.

Second, automatically generated task labels must never be used directly (i.e., "nakedly"). Pseudo-labels generated by large models must undergo a controlled feedback loop for rigorous review.

Finally, data refinement must move towards multimodal joint verification. Future filtering systems must evaluate not only whether the textual description matches the video (semantic dimension) but also whether the trajectory is physically smooth (physical dimension). Only samples where visual evidence and motion evidence corroborate each other can support the generalized learning of next-generation, high-performance robots.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

Report only automatically verified or independently evaluated results. Mark API, Blender, and pending-human evidence explicitly; do not infer completion.

7.2 Future Work

Chapter A

VLM System Prompts

Chapter B

Semantic Consistency Prompt

Chapter C

Data Schemas

Chapter D

Configuration

Chapter E

Additional Results

Chapter F

User Guide